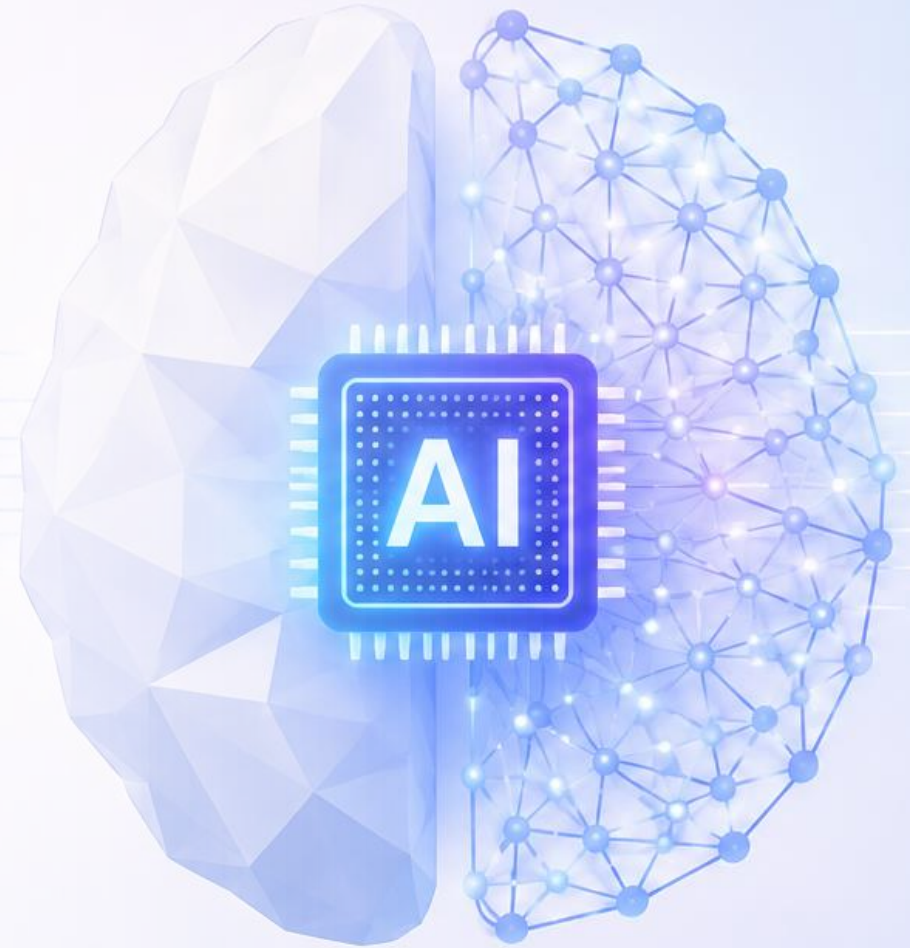


Project 1

Implementation & Grading Guideline



Welcome to Project 1

This project is your opportunity to demonstrate your practical understanding of the core machine learning workflow. Instead of just running black-box algorithms, you will take on the role of a Junior Data Scientist/ML Engineer. You will pick a dataset you have worked with in class, build an optimized machine learning pipeline, write clean code, and defend your design decisions in a live engineering walkthrough.

Assessment Type: Individual Project

Weight: 60% of total grade

Submission Due: Week 9

Live Oral Demonstrations: Week 10 (Allocated timeslots)

Project Goals

- 1. Frame the Problem Space:** Select your data track, identify input features versus target labels, and define whether you are solving a regression or classification problem.
- 2. Engineer the Pipeline:** Process raw structural data, transform unscaled features, and apply machine learning algorithms taught in your course content.
- 3. Optimize & Evaluate:** Systematically tune model parameters and validate generalizability using robust cross-validation techniques.
- 4. Justify Your Engineering:** Defend your architectural choices in writing and during your live presentation, explaining why your approach is optimal for the dataset's unique challenges.



How to get started?

You can choose between **the California Housing** or **the MNIST** datasets depending whether you want to do classification or regression analysis. California dataset is a **regression task where you will predict continuous numerical values (median house prices)**, MNIST dataset is a **multiclass classification task where you will categorize pixel images into discrete labels (digits 0–9)**.

California Housing Dataset



Suitable for Regression Analysis

MNIST Dataset



Suitable for Classification Analysis

How to get started?

You can use google colab to do your project 1. The following code shows how **California Housing** and **MNIST** datasets can be imported in google colab.

California Housing Dataset Loading:

```
from sklearn.datasets import fetch_california_housing

# Loads the data directly into a clean Pandas DataFrame framework
housing_df = fetch_california_housing(as_frame=True).frame
```

MNIST Dataset Loading:

```
from sklearn.datasets import fetch_openml

# Fetches the 70,000 handwritten digit images from OpenML
mnist_data = fetch_openml('mnist_784', version=1, as_frame=False)
```

Regression Analysis

(California Housing Dataset)

Dataset Properties

- **Data Context:** 20,640 census block groups ("districts") across California containing demographic and geographic metrics.
- **Problem Type:** Supervised Regression (Predicting a continuous numerical value).
- **Core Objective:** Predict a district's median house value (MedHouseVal) based on historical census characteristics.

Target & Feature Profile

- **Target Variable:** MedHouseVal
- **Input Features:** Demographic metrics (**MedInc**, **Population**, **HouseAge**), structural counts (**AveRooms**, **AveBedrms**, **AveOccup**), and geographic coordinates (**Latitude**, **Longitude**).

Regression Analysis

(California Housing Dataset)

Step 1: Baseline Modeling Approaches

- **What to do:** Train **two** distinct algorithms with baseline/default settings to establish your performance benchmark.
- **Linear Regression:** Fits a straight baseline hyperplane to test if relationships are simple and linear.
- **Decision Tree Regressor:** Captures non-linear relationships and geographical groupings without complex coordinate changes.
- **Example:** You train a standard Linear Regression model alongside a default Decision Tree Regressor on your training data split, then compare how far off their baseline predictions are using your target house prices.

Step 2: Feature Engineering & Scaling Options

- **What to do:** Clean, normalize, and transform your data rows so your estimators can train efficiently.
- **Feature Scaling:** Apply Standardization (Z-score) or Min-Max scaling to prevent large population counts from overpowering smaller metrics like income.
- **Feature Interaction Metrics:** Combine raw columns to uncover hidden spatial or household trends.
- **Example:** You divide total bedrooms by total rooms to create a new BedroomsPerRoom feature, and use standard scaling to force large columns like population down into a small, uniform range.

Regression Analysis

(California Housing Dataset)

Step 3: Hyperparameter Tuning & Optimization

- **What to do:** Systematically adjust your algorithms' control settings to find the optimal balance between underfitting and overfitting.
- **Regularization Tuning:** Apply Ridge (L2) or Lasso (L1) penalty constraints to prevent linear models from overfitting to noisy variables.
- **Structural Tree Regularization:** Restrict tree complexity by testing boundaries on depth and split properties.
- **Example:** You use an automated grid search to test a specific range of tree depths (like 3, 5, and 10 layers max) to see which setting yields the lowest validation error.

Step 4: Advanced Ensemble Techniques

- **What to do:** Combine the predictions of multiple baseline learners to create a more robust, highly accurate strong learner.
- **Random Forests:** Train an ensemble of independent decision trees using bootstrap sampling (bagging) to drastically reduce variance.
- **Gradient Boosting:** Train a sequence of shallow trees where each new model fixes the residual errors of the previous ones.
- **Stacking Architectures:** Feed predictions from different base models (e.g., Linear Model + Tree) into a meta-regressor for a unified final prediction.
- **Example:** You implement a Random Forest model containing 100 parallel trees that vote on the final price estimation, balancing out individual tree errors.

Regression Analysis

(California Housing Dataset)

(Grading Scheme)

Phase / Step	Pass / Credit (P/C) Requirement	Distinction (D) Extension	High Distinction (HD) Extension
Step 1: Baseline Models	Train 2 basic algorithms using default settings.	Core baseline requirement	Core baseline requirement
Step 2: Feature Engineering	Apply standard scaling (Min-Max/Z-score) and clean missing data.	Calculate custom interaction features (ratios/densities) to uncover deeper trends.	Extract and map Feature Importance scores or coefficients to mathematically justify feature selection.
Step 3: Tuning & Evaluation	Manually adjust basic parameters and report core evaluation metrics.	Implement automated GridSearchCV or RandomizedSearchCV with robust cross-validation.	Perform an end-to-end Leakage-Free Pipeline using Scikit-Learn Pipeline, combined with advanced residual/error analysis.
Step 4: Ensemble Techniques	Not required for P and C	Ensembles are evaluated under HD requirements (if applied appropriately and accurately).	Implement production-grade ensemble methods like Random Forests, Gradient Boosting, or Stacking Architectures .

Classification Analysis

(MNIST Dataset)

Dataset Properties

- **Data Context:** 70,000 grayscale images of handwritten digits (0 through 9) across a flattened pixel layout.
- **Problem Type:** Supervised Multiclass Classification (10 discrete target categories).
- **Core Objective:** Correctly categorize an unseen pixel matrix into its matching numerical digit class.

Target & Feature Profile

- **Target Variable:** Class label (Discrete set: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}).
- **Input Features:** 784 numerical inputs (pixel1 through pixel784), representing the grayscale intensity (0 to 255) of a 28 times 28 image matrix.

Classification Analysis

(MNIST Dataset)

Step 1: Baseline Modeling Approaches

- **What to do:** Train **two** distinct algorithms with baseline/default settings to establish your performance benchmark.
- **SGD Classifier:** A linear classifier trained with stochastic gradient descent that handles high-dimensional pixel layouts quickly.
- **Decision Tree Classifier:** Learns non-linear rule hierarchies to map pixel intensities directly into individual digit classes.
- **Example:** You train a standard SGD Classifier alongside a default Decision Tree Classifier on your training split, then compare their baseline capabilities at identifying digits correctly.

Step 2: Feature Engineering & Scaling Options

- **What to do:** Clean, normalize, and transform your data rows so your estimators can train efficiently.
- **Pixel Intensity Scaling:** Divide raw values by 255 to scale features down from a 0–255 range to a uniform 0.0–1.0 range, stabilizing gradient paths.
- **Dimensionality Reduction (PCA):** Transform the 784-pixel layout into a smaller set of principal components that retain the highest dataset variance while filtering background noise.
- **Example:** You scale your pixel arrays to a 0.0–1.0 span to prevent extreme color values from throwing off your weights, and apply PCA to compress the image data for faster training.

Classification Analysis

(MNIST Dataset)

Step 3: Hyperparameter Tuning & Optimization

- **What to do:** Systematically adjust your algorithms' control settings to find the optimal balance between underfitting and overfitting.
- **Loss Function & Penalty Tuning:** Alternate loss arguments (e.g., comparing 'hinge' vs 'log_loss') and adjust the regularization scale (alpha) to prevent over-memorizing pixel patterns.
- **Threshold & Class Imbalance Adjustments:** Fine-tune decision boundaries along the precision-recall curve or apply class weights to handle over-represented digit groups.
- **Example:** You use an automated grid search to test multiple values for the regularization parameter to see which setting yields the highest validation F1-score.

Step 4: Advanced Ensemble Techniques

- **What to do:** Combine the predictions of multiple baseline learners to create a more robust, highly accurate strong learner.
- **Voting Classifiers:** Combine diverse models (like SGD and Decision Trees) to let their majority vote (Hard) or averaged probabilities (Soft) decide the label.
- **Random Forests:** Train an ensemble of parallel classification trees using bootstrap sampling to stabilize boundaries and minimize classification variance.
- **Stacking Architectures:** Pass predictions from several distinct level-0 classifiers into a final meta-classifier to compute the final digit designation.
- **Example:** You implement a soft-voting classifier that averages the prediction probabilities of your tuned SGD and tree models to make a highly stable combined guess.

Classification Analysis

(MNIST Dataset)

(Grading Scheme)

Phase / Step	Pass / Credit (P/C) Requirement	Distinction (D) Extension	High Distinction (HD) Extension
Step 1: Baseline Models	Train 2 basic algorithms using default settings.	Core baseline requirement	Core baseline requirement
Step 2: Feature Engineering	Apply standard pixel intensity scaling (0.0–1.0 normalization).	Implement Principal Component Analysis (PCA) to compress features and remove pixel noise.	Use PCA to mathematically map Feature Importance/pixel influence , or implement advanced spatial visualizations like t-SNE .
Step 3: Tuning & Evaluation	Report classification accuracy and plot a core confusion matrix.	Implement automated GridSearchCV or RandomizedSearchCV to optimize macro F1-score under cross-validation.	Perform an end-to-end Leakage-Free Pipeline using Scikit-Learn Pipeline, combined with precision-recall threshold adjustments or deep error profile analysis.
Step 4: Ensemble Techniques	Not required for P and C	Ensembles are evaluated under HD requirements (if applied appropriately and accurately).	Implement production-grade ensemble methods like Random Forests, Voting Classifiers, or Stacking Architectures .

Deliverables

- **Deliverable 1 (Python Script):** A complete, well-structured, and error-free implementation of your machine learning pipeline annotated with clear, original comments.
- **Deliverable 2 (Workflow Documentation):** A brief, professional technical report explaining your methods, design choices, and performance evaluation.
- **Deliverable 3 (Live Demonstration):** A live individual walkthrough and oral defense in Week 10 where you execute your code and answer specific technical questions from the assessor.



AI Policy

- **Permitted:** Using AI tools to brainstorm code architecture, outline report sections, or debug stubborn syntax errors.
- **Prohibited:** Generating your entire project using AI. All code comments, performance interpretations, and engineering reports must be your own work. Your live technical defense will flag any work that isn't authentically yours.



Good Luck !

